

Conception Technique

Inventaires_completude_representativite

Boite Eddy

23 août 2026

Résumé

Ce document présente la conception technique du pipeline d'analyse de complétude et de représentativité d'inventaires fongiques `Inventaires_completude_representativite.R` (v1.4). La description adopte un cadre d'ingénierie reproductible : architecture, contrats de données, configuration runtime, robustesse opérationnelle et livrables produits. L'objectif est de fournir une référence techniquement auditable et réutilisable pour l'analyse d'inventaires fongiques par site.

Table des matières

1	Structuration IMRaD adaptée	3
2	Positionnement méthodologique	3
3	Cadre théorique des choix statistiques	3
4	Objectif technique	3
5	Architecture logicielle	4
5.1	Blocs principaux	4
5.2	Fonctions clés	4
5.3	Points d'entrée	4
6	Dépendances techniques	5
6.1	Packages R	5
6.2	Comportement d'installation	5
7	Contrats d'entrée et de sortie	5
7.1	Contrat d'entrée	5
7.2	Configuration runtime (CONFIG)	5
7.3	Modes d'exécution et flags internes	6
7.4	Audit CSV	6
8	Modèle de données technique	6
8.1	Schéma logique des observations	6
8.2	Artefacts intermédiaires	7
9	Pipeline technique détaillé	7
9.1	Phase A — Ingestion et normalisation	7
9.1.1	Critères de qualité de données	8
9.2	Phase B — Indicateurs cumulés et de représentativité	8
9.2.1	Courbe temps-espèces	8
9.2.2	Taux d'espèces exclusives (TEE) et indice de représentativité (Ir)	8

9.3	Phase C — Modélisation et complétude	8
9.4	Phase D — Fréquences, occupation spatiale et CA/AFC	9
9.5	Phase E — Métriques de pertinence scientifique	9
9.6	Phase F — Visualisation	9
10	Résultats	10
10.1	Logging systématisé	10
10.1.1	Fonctions de logging	10
10.1.2	Format et contenu	10
10.2	Organisation thématique des sorties	10
10.3	Journalisation et intégrité des sorties	11
10.4	Livrables	11
10.4.1	Global	11
10.4.2	Par site (<code>sites/<site_name>/</code>)	11
10.5	Flux d'exécution	12
11	Robustesse et gestion d'erreurs	12
11.1	Niveaux de protection	12
11.2	Fallbacks notables	12
11.3	Politique de sévérité des anomalies	13
12	Contrôles qualité et non-régression	13
12.1	Indicateurs d'assurance qualité (QA) réellement exportés	13
12.2	Critères techniques d'acceptation	13
12.3	Stratégie de tests recommandée	13
12.4	Matrice de validation des artefacts	14
12.5	Menaces à la validité	14
13	Complexité et performances (qualitatives)	14
13.1	Pistes d'optimisation	15
14	Sécurité, conformité et exploitation	15
14.1	Sécurité des données	15
14.2	Conformité et bonnes pratiques	15
14.3	Procédure opératoire standard (POS)	15
15	Répliquabilité et transparence scientifique	15
16	Discussion	16
16.1	Améliorations techniques (v1.4)	16
16.1.1	Phase 1 : Fiabilisation	16
16.1.2	Phase 2 : Performance	16
16.1.3	Phase 3 : Graphiques et visualisations	16
16.1.4	Phase 4 : Industrialisation	16
16.2	Limites connues	16
17	Protocole de référence	17
18	Évolutions techniques recommandées	17
19	Références bibliographiques	17
	Contact et versionnement	18

1 Structuration IMRaD adaptée

Dans une perspective de publication scientifique, la lecture du document peut être organisée selon le schéma IMRaD, adapté à une documentation technique :

- **Introduction** : objectifs techniques et périmètre du script.
- **Méthodes** : architecture, contrats d'entrée/sortie, configuration runtime, pipeline.
- **Résultats** : organisation des sorties, livrables et journalisation.
- **Discussion** : limites connues et robustesse.

2 Positionnement méthodologique

Le pipeline relève d'une approche *data-centric* appliquée à l'écologie descriptive de terrain : les indicateurs de complétude et de représentativité sont construits par accumulation successive des observations au fil des visites, puis consolidés par des modèles d'ajustement et des métriques de pertinence agrégées. Deux objectifs sont poursuivis simultanément :

- **validité analytique** (qualité des ajustements de courbes d'accumulation, robustesse des indicateurs TEE/Ir),
- **validité opérationnelle** (stabilité des sorties, traçabilité et exploitabilité décisionnelle par site).

3 Cadre théorique des choix statistiques

Le dispositif s'appuie sur la théorie des courbes d'accumulation d'espèces (*species accumulation curves*) et sur une estimation empirique du taux d'espèces rares. Cette combinaison répond à une double exigence : décrire fidèlement la dynamique de découverte des espèces au fil des visites, et estimer une richesse asymptotique théorique permettant de quantifier objectivement l'effort d'échantillonnage restant.

Dans ce cadre, la sélection des métriques privilégie :

- la **comparabilité inter-sites** (Ir et complétude bornés dans $[0, 1]$),
- la **sensibilité à la stabilisation de la courbe** (pente terminale, part de nouveauté en fin d'inventaire),
- la **robustesse à l'absence de données optionnelles** (dégradation contrôlée si `placette`, `minpack.lm` ou `vegan` sont indisponibles),
- la **traçabilité des décisions** (logs horodatés et manifeste final des livrables).

4 Objectif technique

Ce document décrit l'architecture technique du script `scripts/Inventaires_complétude_representativite.R` (v1.4). Il précise les contrats d'entrée/sortie, la configuration runtime et les limites opérationnelles.

5 Architecture logicielle

5.1 Blocs principaux

1. Initialisation et validation de la configuration (`validate_config()`)
2. Résolution du fichier d'entrée (`resolve_input_file()`)
3. Lecture avec détection automatique du délimiteur (`read_delim_auto()`)
4. Audit de conformité CSV et export du rapport (`audit_csv_conformity()`, `export_csv_conformity_report()`)
5. Préparation et nettoyage des données (`prepare_data()`)
6. Analyse par site (`analyze_site()` appliquée par `purrr::map` sur les groupes de site)
7. Consolidation multi-sites et graphiques comparatifs
8. Manifeste final des livrables (`log_output_manifest()`)

5.2 Fonctions clés

- **Configuration et logging** : `get_embedded_config()`, `setup_logging()`, `log_message()`, `log_info()`, `log_debug()`, `log_warning()`, `log_header()`, `log_section()`
- **Résolution et validation** : `resolve_path()`, `resolve_input_file()`, `validate_config()`, `check_required_packages()`
- **Lecture et audit** : `read_delim_auto()`, `audit_csv_conformity()`, `export_csv_conformity_report()`
- **Préparation** : `prepare_data()`
- **Analyses univariées** : `calc_cumulative_metrics()`, `calc_tee_ir()`, `calc_species_frequency()`, `calc_spatial_occupancy()`
- **Modélisation** : `fit_linear_model()`, `fit_hyperbolic_model()` (asymptote extraite via `extract_model_stats()`)
- **Métriques avancées** : `calc_scientific_metrics()`, `run_ca()`
- **Orchestration** : `analyze_site()`, `run_analysis()`, `log_output_manifest()`, `ensure_thematic_dirs()`
- **Visualisation** : `theme_project()`, `PALETTE_PRIMARY` (5 teintes CVD-friendly), `save_plot()`, fonctions `plot_*()`

5.3 Points d'entrée

- Le lancement automatique est conditionné par le flag `AUTO_RUN` (piloté par `INVENTAIRES_AUTO_RUN`) : en session non interactive (`Rscript`), `run_analysis(CONFIG)` est appelé directement ; en session interactive, un message informe que la fonction est prête à être invoquée manuellement.
- Contrairement au script CHEGD de référence, aucun bloc `tryCatch` global n'encapsule `run_analysis()` : une erreur non capturée localement (ex. `stop()` de `validate_config()` ou `prepare_data()`) interrompt l'exécution complète (voir § 11).

6 Dépendances techniques

6.1 Packages R

- Requis : `dplyr`, `tidyr`, `ggplot2`, `purrr`, `readr`, `stringr`, `forcats`, `tibble`, `scales`, `gridExtra`.
- Optionnels : `minpack.lm` (nlsLM, modèle hyperbolique), `vegan` (cca, CA/AFC).

6.2 Comportement d'installation

`check_required_packages()` vérifie la disponibilité de chaque package requis via `requireNamespace()`. En cas d'absence, le script s'arrête avec une erreur explicite et une commande `install.packages()` suggérée (pas d'installation automatique). Les packages optionnels sont détectés via les indicateurs `HAS_VEGAN`/`HAS_MINPACK` sans bloquer l'exécution : les fonctionnalités correspondantes sont simplement désactivées avec un message `info` explicite.

7 Contrats d'entrée et de sortie

7.1 Contrat d'entrée

`data/observations.csv` (ou `.txt/.tsv`) avec colonnes obligatoires `site`, `date`, `visite_id`, `espece` et optionnelle `placette`.

Règles : visite distincte `date + visite_id` (et `site` en multi-sites), parsing date strict, déduplication de la clé observation.

7.2 Configuration runtime (CONFIG)

Clé	Défaut	Description
<code>input_file</code>	<code>data/observations.csv</code>	Fichier source
<code>output_dir</code>	<code>results/ICR</code>	Répertoire de sortie
<code>date_format</code>	<code>%Y-%m-%d</code>	Parsing des dates
<code>csv_strict_mode</code>	<code>TRUE</code>	Arrêt si CSV non conforme
<code>csv_allow_extra_cols</code>	<code>FALSE</code>	Tolérance des colonnes supplémentaires
<code>csv_required_cols</code>	<code>site, date, visite_id, espece</code>	Schéma obligatoire
<code>csv_optional_cols</code>	<code>placette</code>	Schéma optionnel
<code>freq_breaks</code>	<code>0, 0.10, 0.25, 0.50, 0.75, 1.00</code>	Séquence des intervalles de fréquence (proportions de visites, dans <code>[0, 1]</code>)
<code>freq_labels</code>	<code>exceptionnelle, très_rare, occasionnelle, fréquente, constante</code>	Étiquettes des intervalles
<code>make_ca</code>	<code>TRUE</code>	Active la CA/AFC si possible
<code>min_visits_for_model</code>	<code>5</code>	Seuil de modélisation
<code>width</code>	<code>10</code>	Largeur graphiques PNG (pouces)
<code>height</code>	<code>6</code>	Hauteur graphiques PNG (pouces)

dpi	300	Résolution (points/pouce)	graphiques	PNG
-----	-----	------------------------------	------------	-----

Seules cinq clés sont effectivement surchargeables via variables d'environnement dans `get_embedded_config()` : `input_file`, `output_dir`, `date_format`, `csv_strict_mode` et `csv_allow_extra_cols`. Les clés `freq_breaks`, `freq_labels`, `make_ca`, `min_visits_for_model`, `width`, `height` et `dpi` sont actuellement des constantes du code source (littéraux R) : leur modification requiert d'éditer la fonction `get_embedded_config()` elle-même, malgré le principe de configuration centralisée. La configuration effective est affichée dans l'en-tête du script via la fonction `print_startup_header()`.

Variables d'environnement effectivement lues : `INVENTAIRES_INPUT_FILE`, `INVENTAIRES_OUTPUT_DIR`, `INVENTAIRES_DATE_FORMAT`, `INVENTAIRES_CSV_STRICT`, `INVENTAIRES_CSV_ALLOW_EXTRA_COLS` et `INVENTAIRES_AUTO_RUN`. Les autres options reprennent les clés de configuration.

7.3 Modes d'exécution et flags internes

Le script expose des flags globaux :

- `DEBUG_MODE` (défaut `FALSE`) : warnings affichés en continu et logs détaillés.
- `BENCHMARK_MODE` (défaut `FALSE`) : chronométrage des blocs de calcul.
- `CLEAN_ENVIRONMENT` (défaut `FALSE`) : purge de l'environnement avant lancement (mode expert).

Le lancement automatique est contrôlé via `INVENTAIRES_AUTO_RUN` (défaut `TRUE`).

7.4 Audit CSV

Sorties d'audit (sous `results/ICR/00_data_quality/`) :

- `ICR_00_csv_conformite_report.csv`
- `ICR_00_csv_conformite_problems.csv` (si anomalies de parsing)

En mode strict (`csv_strict_mode = TRUE`, défaut), toute non-conformité (colonne manquante, colonne supplémentaire non tolérée, problème de parsing) est bloquante (`stop()`). En mode tolérant (`FALSE`), un `log_warning()` est émis et le pipeline se poursuit sur les données telles que lues (voir §9).

8 Modèle de données technique

8.1 Schéma logique des observations

Champ	Type	Rôle
site	caractère	Clé de groupement principale ; détermine le découpage <code>analyze_site()</code> et le sous-répertoire <code>sites/<slug>/</code> .
date	date (par-sée selon <code>date_format</code>)	Ordonnancement chronologique des visites ; composant de la clé de déduplication.
visite_id	caractère	Combiné à <code>date</code> pour définir une visite distincte (<code>count_distinct_visits()</code>).

espece	caractère (normalisé via <code>stringr::str_squish()</code>)	Unité d'accumulation pour toutes les courbes et indices (TEE, Ir, fréquence).
placette	caractère, optionnel	Active <code>calc_spatial_occupancy()</code> et <code>run_ca()</code> si renseignée (non totalement NA/vide) ; sinon les sorties correspondantes sont omises (statut OPTIONNEL_NON_GENERE).

Clé de déduplication appliquée par `prepare_data()` : (site, placette, date, visite_id, espece).

8.2 Artefacts intermédiaires

- **Données préparées** : table nettoyée/dédupliquée, exportée telle quelle (`ICR_donnees_preparees.csv`).
- **Table des visites** (`visits_tbl`) : visites distinctes `date` + `visite_id`, triées chronologiquement, indexées par `visite_index`.
- **Table cumulée** (`df_cum`) : une ligne par visite avec `nb_especes_trouvees`, `nb_nouvelles`, `nb_cumule`.
- **Table TEE/Ir** (`df_tee`) : une ligne par visite avec `tee` et `ir`.
- **Table de fréquences** (`freq_tbl`) : une ligne par espèce avec `prop_visites` et classe qualitative.
- **Table d'occupation spatiale** (`occ_tbl`, optionnelle) : une ligne par espèce avec `prop_placettes`.
- **Statistiques de modèles** (`model_stats`) : R^2 , équation et asymptote pour les modèles linéaire et hyperbolique.
- **Métriques scientifiques** (`scientific_metrics`) : agrégats par site issus de `calc_scientific_metrics()`.
- **Consolidations globales** : `site_summaries` et `site_metrics`, obtenues par `purrr::map()` + `dplyr::bind_rows()` sur les résultats de chaque site.

9 Pipeline technique détaillé

9.1 Phase A — Ingestion et normalisation

- `resolve_input_file()` : tente le chemin configuré (`input_file/INVENTAIRES_INPUT_FILE`) ; à défaut, recherche un fichier `observations.{csv,txt,tsv}` sous `data/` ; `stop()` explicite si aucun candidat trouvé.
- `read_delim_auto()` : détecte le délimiteur en comptant les occurrences de tabulation, point-virgule et virgule sur la première ligne, puis lit via `readr::read_delim()` (locale UTF-8, toutes colonnes en caractère par défaut via `build_csv_colspec()`) ; les problèmes de parsing sont capturés via `readr::problems()`.
- `audit_csv_conformity()` : calcule colonnes manquantes/supplémentaires, nombre de problèmes de parsing et statut CONFORME/NON_CONFORME ; `export_csv_conformity_report()` écrit le rapport dans `00_data_quality/`.

- `prepare_data()` : vérifie la présence des colonnes obligatoires, normalise les champs texte, parse les dates, `stop()` explicite si valeurs manquantes/vides sur `site/visite_id/espece/date`, ajoute `placette = NA_character_` si absente, déduplique sur la clé composée et journalise les effectifs avant/après.

9.1.1 Critères de qualité de données

- Complétude des colonnes obligatoires (arrêt si absentes).
- Validité typologique des dates (arrêt si non parsées selon `date_format`).
- Traçabilité des doublons supprimés (nombre de lignes avant/après déduplication journalisé).

9.2 Phase B — Indicateurs cumulés et de représentativité

9.2.1 Courbe temps-espèces

`calc_cumulative_metrics()` trie les visites chronologiquement (`date`, `visite_id`), leur attribue un `visite_index`, puis parcourt chaque visite i pour calculer :

- `nb_especes_trouvees` : nombre d'espèces observées lors de la visite i ,
- `nb_nouvelles` : cardinal de la différence ensembliste entre les espèces de la visite i et l'ensemble des espèces déjà vues (`setdiff()`),
- `nb_cumule` : cardinal de l'union cumulative des espèces vues jusqu'à i (`union()`).

9.2.2 Taux d'espèces exclusives (TEE) et indice de représentativité (Ir)

`calc_tee_ir()` considère, pour chaque visite i , l'ensemble des espèces vues sur les visites 1 à i :

$$TEE_i = \frac{s_{1,i}}{s_{total,i}} \quad Ir_i = 1 - TEE_i$$

où $s_{1,i}$ est le nombre d'espèces vues exactement une fois parmi les visites 1.. i , et $s_{total,i}$ le nombre d'espèces distinctes vues parmi les visites 1.. i . Repères de lecture utilisés dans les graphiques : $Ir < 0.60$ (faible), $0.60 \leq Ir < 0.80$ (moyenne), $Ir \geq 0.80$ (bonne).

9.3 Phase C — Modélisation et complétude

- **Modèle linéaire** : `fit_linear_model()` ajuste `stats::lm(nb_cumule ~ visite_index)`.
- **Modèle hyperbolique** (si `HAS_MINPACK`) : `fit_hyperbolic_model()` ajuste, via `minpack.lm::nlsLM` (Levenberg-Marquardt, 500 itérations max), l'équation

$$S(t) = a - \frac{1}{bt + c}$$

avec valeurs de départ $a_0 = 1.20 \times \max(\text{nb_cumule})$, $b_0 = 10^{-4}$, $c_0 = 10^{-3}$.

- **Extraction des statistiques** : `extract_model_stats()` calcule $R^2 = 1 - SS_{res}/SS_{tot}$; pour le modèle hyperbolique, l'asymptote verticale est a et l'asymptote horizontale (abscisse théorique d'origine) est $-c/b$.
- **Complétude** : `completude = nb_cumule_max/a` (asymptote hyperbolique), non calculée (NA) si a est absent ou ≤ 0 . Repères : < 0.70 (insuffisante), $0.70-0.90$ (intermédiaire), ≥ 0.90 (avancée).
- **Condition d'activation** : modélisation déclenchée uniquement si `nrow(df_cum) >= config$min_visits_for_model` (défaut 5).

9.4 Phase D — Fréquences, occupation spatiale et CA/AFC

- `calc_species_frequency()` : `prop_visites = nb_visites/n_visites_totales`, discrétisé via `cut(freq_breaks, freq_labels, include.lowest = TRUE, right = TRUE)`.
- `calc_spatial_occupancy()` : requiert une colonne `placette` non entièrement NA/vide ; `prop_placettes = nb_placettes/n_placettes_totales` ; retourne NULL sinon.
- `run_ca()` : requiert HAS_VEGAN et des données de placette ; construit une matrice binaire placette $\times$ espèce (`tidyr::pivot_wider()`), exige au moins 3 colonnes d'espèces, puis appelle `vegan::cca()`.

9.5 Phase E — Métriques de pertinence scientifique

`calc_scientific_metrics()` calcule, par site :

- **Pente terminale** (`slope_terminal_cumul`) : pente d'une régression linéaire sur la fenêtre terminale de $k = \min(5, n_{visites})$ points de la courbe cumulée.
- **Part de nouveauté terminale** (`tail_novelty_share`) : $\sum nb_nouvelles_{fen\hat{e}tre} / \sum nb_nouvelles_{total}$.
- **TEE/Ir finaux** : dernière valeur (`dplyr::last()`) des séries `tee/ir`.
- **R^2 retenu** : celui du modèle hyperbolique si disponible, sinon celui du modèle linéaire.
- **Corrélation temporel/spatial** (`spearman_temporal_spatial`) : corrélation de Spearman (`stats::cor(method = "spearman")`) entre `prop_visites` et `prop_placettes` pour les espèces communes à `freq_tbl` et `occ_tbl`, calculée uniquement si $n \geq 3$ espèces jointes.
- **Taux de discordance** (`discordance_rate`) : proportion d'espèces pour lesquelles $|\text{prop_visites} - \text{prop_placettes}| > 0.50$.
- **Score de pertinence** (`score_pertinence`) : moyenne pondérée

$$score = 0.35 \cdot Ir_{final} + 0.35 \cdot completude + 0.20 \cdot (1 - tail_novelty_share) + 0.10 \cdot R^2_{mod\grave{e}le}$$

les composantes manquantes (NA) sont exclues et les poids restants renormalisés.

- **Classes qualitatives** : `class_ir` (< 0.60 faible, < 0.80 moyenne, ≥ 0.80 bonne) et `class_completude` (< 0.70 insuffisante, < 0.90 intermédiaire, ≥ 0.90 avancée).

9.6 Phase F — Visualisation

Toutes les figures partagent `theme_project()` et la palette `PALETTE_PRIMARY` (bleu #0072B2, orange #E69F00, vert #009E73, rouge #D55E00, violet #CC79A7 — palette adaptée au daltonisme). Fonctions principales : `plot_richness_over_time()`, `plot_time_species_curve()` (annotation de l'asymptote hyperbolique), `plot_tee_ir()` (deux panneaux via `gridExtra::grid.arrange()`, zones de couleur $< 0.60/0.60\text{--}0.80/\geq 0.80$), `plot_frequency_hist()`, `plot_spatial_vs_temporal()` (conditionnelle), `plot_scientific_metrics_site/heatmap/score()`. `save_plot()` exporte via `ggplot2::ggsave()` (format PNG dans tous les appels du script, `width/height/dpi` configurés), avec `tryCatch` et `log_warning()` en cas d'échec d'écriture.

10 Résultats

10.1 Logging systématisé

Chaque exécution génère un fichier log horodaté **sous logs/ à la racine du projet** (PROJECT_DIR/logs/, où PROJECT_DIR est le répertoire parent du dossier **scripts/**) — et non sous **results/ICR/** comme les autres livrables, puisque `setup_logging()` est appelé avec `base_dir = PROJECT_DIR`.

10.1.1 Fonctions de logging

- `setup_logging(base_dir, prefix)` : Initialise l'environnement `.log_env` et crée le répertoire `logs/` ; appelée au démarrage
- `log_message(level, ...)` : Fonction centrale avec gestion intelligente des format strings
- `log_info(...)` : Wrapper niveau INFO
- `log_debug(...)` : Wrapper niveau DEBUG
- `log_warning(...)` : Wrapper niveau WARN (affiche aussi en `stderr`)
- `log_header(script_name, script_version)` : Affiche l'en-tête du script avec métadonnées
- `log_section(title)` : Affiche un titre de section avec délimiteurs

10.1.2 Format et contenu

- **Format des entrées** : [YYYY-MM-DD HH:MM:SS] [LEVEL] message
- **Nom du fichier** : PROJECT_DIR/logs/ICR_YYYYMMDD_HHMM.log
- **Contenu** : En-tête du script, métadonnées (timestamp, R version, working dir), progression du pipeline, manifeste final des livrables

10.2 Organisation thématique des sorties

Les résultats sont organisés selon une hiérarchie prédéfinie sous **results/ICR/** (créée par `ensure_thematic_dirs()`) ; les noms exacts des répertoires réservés (`04_modeles`, `05_spatial`) sont ceux définis dans le code source :

```
results/ICR/
+-- 00_data_quality/
|   +-- ICR_00_csv_conformite_report.csv
|   +-- ICR_00_csv_conformite_problems.csv (si anomalies)
|   '-- ICR_donnees_preparees.csv
+-- 01_richesse/ (réservé pour extensions)
+-- 02_completude_repr/ (réservé)
+-- 03_frequences/ (réservé)
+-- 04_modeles/ (réservé)
+-- 05_spatial/ (réservé)
+-- 06_metrics/
|   +-- ICR_metrics_pertinence_tous_sites.csv
|   +-- ICR_metrics_pertinence_heatmap_sites.png
|   '-- ICR_metrics_pertinence_score_sites.png
+-- comparisons/
|   +-- ICR_resume_tous_sites.csv
```

```
|   +-- ICR_comparaison_completude_sites.png
|   '-- ICR_comparaison_ir_sites.png
'-- sites/
    +-- <site_name>/
    |   +-- ICR_01_courbe_temps_especes.csv
    |   +-- ICR_02_tee_ir.csv
    |   +-- ICR_03_frequence_especes.csv
    |   +-- ICR_04_resume_site.csv
    |   +-- ICR_07_metrics_pertinence.csv
    |   +-- ICR_01_richesse_par_visite_et_cumul.png
    |   +-- ICR_02_courbe_temps_especes_hyperbole.png
    |   +-- ICR_03_tee_ir.png
    |   +-- ICR_04_histogramme_frequences.png
    |   +-- ICR_07_metrics_pertinence_dashboard.png
    |   +-- ICR_05_modeles.csv (si modèles activés)
    |   +-- ICR_06_occupation_spatiale.csv (si placette)
    |   +-- ICR_05_temporel_vs_spatial.png (conditionnelle)
    |   '-- ICR_06_CA_placettes_especes.png (si vegan + placette)
    '-- <other_site_name>/
        '-- ...
```

```
PROJECT_DIR/logs/
'-- ICR_YYYYMMDD_HHMM.log
```

10.3 Journalisation et intégrité des sorties

La journalisation est réalisée en console via `log_info()`, `log_debug()` et `log_warning()`.

En fin de pipeline, `log_output_manifest()` contrôle les livrables avec statuts : OK, MANQUANT, OPTIONNEL_NON_GENERE.

Le script supprime automatiquement tout `Rplots.pdf` parasite généré pendant l'exécution (niveau site et global).

10.4 Livrables

10.4.1 Global

- **Qualité des données** (`00_data_quality/`) : `ICR_00_csv_conformite_report.csv`, `ICR_00_csv_conformite_problems.csv` (optionnel), `ICR_donnees_preparees.csv`.
- **Comparaisons** (`comparisons/`) : `ICR_resume_tous_sites.csv`, `ICR_comparaison_completude_sites.png`, `ICR_comparaison_ir_sites.png`.
- **Métriques** (`06_metrics/`) : `ICR_metrics_pertinence_tous_sites.csv`, `ICR_metrics_pertinence_heatmap_sites.png`, `ICR_metrics_pertinence_score_sites.png`.

Logs : `ICR_YYYYMMDD_HHMM.log` (sous `PROJECT_DIR/logs/`, hors de `results/ICR/`).

10.4.2 Par site (`sites/<site_name>/`)

Fichiers CSV par site : courbe temps-espèces, TEE/IR, fréquences, résumé de site et métriques de pertinence, avec les graphiques PNG associés.

Sorties conditionnelles : `ICR_05_modeles.csv`, `ICR_06_occupation_spatiale.csv`, `ICR_05_temporel_vs_spatial.png`, `ICR_06_CA_placettes_especes.png`.

10.5 Flux d'exécution

```
run_analysis(CONFIG)
-> validate_config()
-> resolve_input_file()
-> read_delim_auto()
-> audit_csv_conformity()
-> export_csv_conformity_report()
-> prepare_data()
-> for each site: analyze_site()
    -> calculs cumul/TEE/frequences/spatial
    -> modeles lineaire/hyperbolique
    -> metriques scientifiques
    -> exports CSV/PNG
-> consolidation multi-sites
-> graphiques globaux
-> log_output_manifest()
```

11 Robustesse et gestion d'erreurs

11.1 Niveaux de protection

- `validate_config()` : vérifie la présence et le type de chacune des clés de configuration (`stop()` explicite sinon), notamment la monotonie de `freq_breaks`, la cohérence de longueur avec `freq_labels` et `min_visits_for_model` ≥ 2 .
- `prepare_data()` : contrôle du schéma et des valeurs manquantes/vides sur `site/visite_id/espece/date` (`stop()` explicite, message ciblant la colonne en cause).
- `tryCatch` locaux dans `analyze_site()` : chaque tentative d'ajustement (`fit_linear_model()`, `fit_hyperbolic_model()`) et d'analyse CA/AFC (`run_ca()`) est isolée ; un échec est journalisé via `log_warning()` et n'interrompt ni le site ni le pipeline.
- `tryCatch` sur chaque écriture CSV (`readr::write_csv()`) : un échec d'écriture provoque un `stop()` explicite mentionnant le fichier concerné (contrairement aux échecs de modélisation, jugés non recouvrables).
- **Absence de `tryCatch` global** : à la différence du script CHEGD de référence (qui encapsule son point d'entrée principal), `run_analysis()` n'est pas protégée globalement ; toute erreur de validation amont (configuration, CSV non conforme en mode strict, données invalides) interrompt l'exécution complète du script.

11.2 Fallbacks notables

- Package `minpack.lm` absent $\Rightarrow$ pas de modèle hyperbolique ni de complétude (valeurs NA), message `log_info()` explicite au démarrage.
- Package `vegan` absent, ou colonne `placette` absente/vide $\Rightarrow$ pas de CA/AFC (`ICR_06_CA_placettes_especes.png` non généré).
- `nb_visites < min_visits_for_model` $\Rightarrow$ pas de modélisation pour le site concerné (`ICR_05_modeles.csv` non généré).
- Absence de colonne `placette` $\Rightarrow$ pas d'occupation spatiale (`occ_tbl` NULL), pas de graphique spatio-temporel, et corrélation `spearman_temporal_spatial` non calculée.

11.3 Politique de sévérité des anomalies

Niveau	Déclencheurs typiques	Comportement effectif
INFO	Package optionnel absent, mode d'exécution, volumétrie des données	Message informatif, aucune interruption
ALERTE	CSV non conforme en mode tolérant, échec d'ajustement de modèle, échec CA/AFC	<code>log_warning()</code> (console + fichier + <code>stderr</code>), le pipeline se poursuit avec livrable(s) correspondant(s) marqué(s) <code>OPTIONNEL_NON_GENERE</code>
ERREUR	Colonne obligatoire absente, valeurs manquantes sur les champs clés, CSV non conforme en mode strict, échec d'écriture d'un CSV	<code>stop()</code> : arrêt immédiat de l'exécution

12 Contrôles qualité et non-régression

12.1 Indicateurs d'assurance qualité (QA) réellement exportés

- `ICR_00_csv_conformite_report.csv` : statut `CONFORME/NON_CONFORME`, colonnes manquantes/supplémentaires, nombre de problèmes de parsing.
- `ICR_00_csv_conformite_problems.csv` (optionnel) : détail des problèmes `readr::problems()`.
- Manifeste des livrables (`log_output_manifest()`) : statuts `OK/MANQUANT/OPTIONNEL_NON_GENERE` par fichier attendu, journalisés en fin d'exécution.
- `class_ir/class_completude` dans `ICR_07_metrics_pertinence.csv` : classes qualitatives directement exploitables pour un contrôle de cohérence.

12.2 Critères techniques d'acceptation

1. Exécution sans erreur fatale pour un CSV conforme au schéma attendu.
2. Génération complète des sorties CSV/figures attendues par site et au global (manifeste sans statut `MANQUANT` non justifié).
3. Log horodaté présent sous `PROJECT_DIR/logs/`.
4. Cohérence des indicateurs TEE/Ir/complétude avec leurs seuils documentés (bornes `[0, 1]`).

12.3 Stratégie de tests recommandée

- **Tests unitaires** : `calc_tee_ir()`, `calc_cumulative_metrics()`, `calc_species_frequency()` et `calc_scientific_metrics()` sur des jeux synthétiques à richesse connue (valeurs de TEE/Ir/score calculables à la main).
- **Tests d'intégration** : exécution complète sur un jeu de données réduit, avec et sans colonne placette.
- **Tests de non-régression** : comparaison des CSV clés (`ICR_04_resume_site.csv`, `ICR_07_metrics_pertinence.csv`) avec des instantanés de référence versionnés.
- **Tests de robustesse** : dates ambiguës, doublons volontaires, sites avec un nombre de visites proche ou inférieur à `min_visits_for_model`, absence simulée de `minpack.lm/vegan`.

12.4 Matrice de validation des artefacts

Artefact	Règle de validation	Preuve attendue
Ingestion (ICR_donnees_preparation)	Aucune valeur manquante sur <code>site_id/espece/date</code>	Comptage avant/après déduplication journalisé
Courbe temps-espèces (ICR_01)	<code>nb_cumule</code> croissant (ou stable) avec <code>visite_index</code>	Contrôle de monotonie
TEE/Ir (ICR_02)	$TEE_i, Ir_i \in [0, 1]$ pour toute visite i	Bornage vérifiable directement sur le CSV
Fréquences (ICR_03)	Classes cohérentes avec <code>freq_breaks/freq_labels</code>	Recoupement <code>cut()</code> manuel sur <code>prop_visites</code>
Modèles (ICR_05, si actifs)	$R^2 \in [0, 1]$, asymptote $> \max(\text{nb_cumule})$	Comparaison directe aux données observées
Pertinence scientifique (ICR_07)	<code>score_pertinence</code> $\in [0, 1]$, classes <code>class_ir/class_completude</code> cohérentes avec les seuils documentés	Recalcul manuel de la moyenne pondérée

12.5 Menaces à la validité

- **Interne** : dépendance forte à la qualité de saisie de `visite_id/date` (une erreur de saisie fusionne ou scinde artificiellement des visites).
- **Externe** : transférabilité limitée si le protocole d'échantillonnage (nombre de visites, granularité de `placette`) diffère fortement d'un site à l'autre ; les comparaisons inter-sites doivent en tenir compte.
- **De conclusion** : prudence sur la complétude/Ir des sites dont le nombre de visites est proche du seuil `min_visits_for_model`, où les estimations restent statistiquement fragiles.

13 Complexité et performances (qualitatives)

- `calc_cumulative_metrics()/calc_tee_ir()` : implémentées via une boucle explicite sur les visites (`for` sur `seq_len(nrow(visits_tbl))`) avec un filtrage cumulatif à chaque itération, soit un coût approximativement $O(v^2)$ par site dans le pire cas (v = nombre de visites) — acceptable pour les volumes usuels d'un inventaire de terrain, mais à surveiller pour des séries de visites très longues.
- Le commentaire d'en-tête du script (Phase 2, v1.2) mentionne une vectorisation des calculs cumulés ; l'implémentation actuelle conserve toutefois une boucle explicite par visite pour ces deux fonctions — écart entre intention documentée et code effectif à conserver à l'esprit lors d'une future refactorisation.
- `calc_species_frequency()/calc_spatial_occupancy()` : agrégations vectorisées `dplyr`, coût proche de $O(n)$ sur le nombre d'observations.
- `fit_hyperbolic_model()` : coût dominé par les itérations de Levenberg-Marquardt (500 au maximum), généralement négligeable devant les boucles ci-dessus pour les tailles de site habituelles.
- `BENCHMARK_MODE` : `bench_time()` chronomètre `calc_cumulative_metrics()`, `calc_tee_ir()` et `calc_species_frequency()` par site lorsqu'activé.

13.1 Pistes d’optimisation

- Vectoriser `calc_cumulative_metrics()/calc_tee_ir()` (cumul par opérations ensemblistes vectorisées plutôt que boucle explicite) pour les sites à très grand nombre de visites.
- Réutiliser `freq_tbl/occ_tbl` déjà calculées comme entrées directes de `calc_scientific_metrics()` sans recalcul intermédiaire.
- Limiter la recréation des répertoires thématiques réservés lorsqu’ils restent vides d’une exécution à l’autre.

14 Sécurité, conformité et exploitation

14.1 Sécurité des données

Le script opère exclusivement sur des fichiers locaux (aucun appel réseau, aucune transmission de données). Les journaux et les CSV générés peuvent contenir des noms de sites potentiellement sensibles (localisation d’espèces protégées) : leur diffusion doit suivre la politique de confidentialité du projet.

14.2 Conformité et bonnes pratiques

- Aucune dépendance à des secrets ou identifiants (pas de connexion externe).
- Chemins résolus de manière relative au projet (`resolve_path()`), limitant les risques de chemins absolus non maîtrisés.
- Traçabilité des sorties via le manifeste (`log_output_manifest()`) et les logs horodatés.

14.3 Procédure opératoire standard (POS)

1. Vérifier la présence et le schéma du fichier d’entrée (`data/observations.csv`).
2. Exécuter le script (`Rscript` ou session interactive).
3. Contrôler le log généré et le rapport de conformité CSV.
4. Diffuser les exports CSV/PNG validés aux parties prenantes.
5. Archiver le run (log + rapport de conformité + CSV consolidés) pour traçabilité.

15 Réplicabilité et transparence scientifique

Le script ne génère pas actuellement de manifeste de run structuré (type JSON) ; seul le manifeste texte des livrables (`log_output_manifest()`) et le fichier log horodaté assurent la traçabilité. Pour renforcer la répliquabilité inter-campagnes, un manifeste de run `run_manifest.json` pourrait consigner :

- `run_id`, `timestamp_start`, `timestamp_end` ;
- `r_version`, versions des packages `dplyr/ggplot2/minpack.lm/vegan` ;
- `input_file`, empreinte (hash) du fichier d’entrée ;
- `config_effective` (valeurs réelles de `CONFIG` après surcharge par variables d’environnement) ;
- `outputs_generated` (liste des fichiers effectivement produits) ;
- `qa_summary` (statut de conformité CSV, nombre de sites traités).

16 Discussion

16.1 Améliorations techniques (v1.4)

16.1.1 Phase 1 : Fiabilisation

- Détection automatique du délimiteur (CSV/TSV/TXT)
- Parsing flexible des dates via format configurable
- Déduplication multi-clé : (site, placette, date, visite_id, espece)
- Gestion granulaire des erreurs CSV (colonne manquante, type invalide, etc.)

16.1.2 Phase 2 : Performance

- Optimisation du modèle hyperbolique : `minpack.lm::nlsLM` avec détection de convergence
- Calculs cumulés intentionnellement documentés comme vectorisés dans l'en-tête du script ; l'implémentation effective conserve une boucle explicite par visite (voir § 11 et la section Complexité et performances)
- Mémorisation des graphiques ggplot2 avant export PNG
- Cache des analyses de CA/AFC conditionnelle

16.1.3 Phase 3 : Graphiques et visualisations

- Cartes de chaleur multi-sites (heatmap) avec échelle de couleur
- Tableaux de bord par site (7 graphiques dans une grille)
- Comparaisons visuelles de complétude (barplot multi-sites)
- Analyses de CA/AFC si `vegan` disponible et placette présente

16.1.4 Phase 4 : Industrialisation

1. **Configuration centralisée** : `get_embedded_config()` avec 14+ paramètres
2. **Logging systématisé** : `setup_logging()` + fonctions `log_*()` avec fichiers horodatés
3. **Organisation thématique** : hiérarchie logique `/sites/`, `/06_metrics/`, `/00_data_quality/`, `/comparisons/`
4. **Métriques avancées** : calcul de pertinence scientifique, dashboards analytiques, manifeste final

16.2 Limites connues

- Sans placette, pas d'analyse spatiale ni de CA/AFC, et la corrélation temporel/spatial de `calc_scientific_metrics()` n'est pas calculée.
- Sans `minpack.lm`, pas d'asymptote hyperbolique ni de complétude associée ; le score de pertinence se rabat alors sur le R^2 du modèle linéaire.
- Si `nb_visites < min_visits_for_model`, pas de modélisation pour le site (impact direct sur `completude` et `score_pertinence`).
- En mode strict (défaut), un CSV non conforme bloque tout le pipeline sans possibilité de reprise partielle.

- Absence de `tryCatch` global autour de `run_analysis()` : une erreur de configuration ou de données en amont interrompt l'exécution complète plutôt que de permettre un traitement partiel des sites valides.
- Les clés `freq_breaks`, `freq_labels`, `make_ca`, `min_visits_for_model`, `width`, `height` et `dpi` ne sont pas surchargeables par variable d'environnement, contrairement à ce que suggère le principe de configuration centralisée.

17 Protocole de référence

Les indicateurs implémentés (courbe temps-espèces, taux d'espèces exclusives, indice de représentativité) s'appuient sur les principes présentés dans le cours Cours 24 Inventaires du Diplôme Universitaire de Mycologie 2026 (Professeur Pierre-Arthur Moreau, Université de Lille, UFR3S PHAR, diapositives 25 à 48), qui formalise la logique d'accumulation d'espèces au fil des visites et l'estimation de la représentativité d'un inventaire de terrain. Contrairement au protocole CHEGD de Sellier (module fiabilité du script de référence), aucun protocole normatif externe spécifique n'est requis ici : le script est entièrement autonome et ne dépend d'aucun classeur ou référentiel externe.

18 Évolutions techniques recommandées

- Ajouter un `tryCatch` global autour de `run_analysis()` pour aligner la robustesse sur le script CHEGD de référence et éviter l'arrêt complet sur une erreur isolée.
- Vectoriser `calc_cumulative_metrics()/calc_tee_ir()` pour supprimer la boucle explicite par visite et réduire la complexité quadratique observée.
- Exposer `freq_breaks/freq_labels/min_visits_for_model` par variable d'environnement, en cohérence avec le principe de configuration centralisée déjà appliqué aux autres clés.
- Générer un manifeste de run structuré (`run_manifest.json`) pour la traçabilité inter-campagnes (voir § Réplicabilité et transparence scientifique).
- Ajouter une suite de tests unitaires sur les formules métier (TEE, Ir, complétude, score de pertinence) afin de sécuriser les futures refactorisations.

19 Références bibliographiques

- Moreau, P.-A. *Cours 24 — Inventaires*. Diplôme Universitaire de Mycologie 2026, Université de Lille (UFR3S PHAR), diapositives 25–48.
- Colwell, R.K. & Coddington, J.A. (1994). Estimating terrestrial biodiversity through extrapolation. *Philosophical Transactions of the Royal Society B*, 345(1311), 101–118.
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer.
- Wickham, H., & Grolemund, G. (2017). *R for Data Science*. O'Reilly.
- Oksanen, J. et al. *vegan: Community Ecology Package*. Paquet R (analyse de correspondance CA/AFC).
- Elzhov, T.V., Mullen, K.M., Spiess, A.-N., & Bolker, B. *minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm*. Paquet R.

Auteur	Eddy Boite
Dépôt GitHub	eddyboite-nat/myco_apps_releases
Branche	main
Version script	1.4
Date de ce document	9 juillet 2026

Contact et versionnement

Pour toute évolution du script, ouvrir une issue dans le dépôt GitHub ou documenter les changements dans le journal `CHANGELOG.md` du projet.